

Recordatorio

En las iteraciones anteriores se implementó:

- **Iteración 01:** programa en consola (CLI) dentro de un único archivo, versión que sirvió entender la mecánica básica de la ruleta.
- **Iteración 02:** ejercicio de uso de ventanas para el módulo de login y registro en Swing, con usuarios hardcoded y almacenamiento en memoria.
- **Iteración 03:** aplicación del Principio de Responsabilidad Única (SRP), separando la lógica de la ruleta en clases.
- **Iteración 04:** incorporación del patrón Modelo–Vista–Controlador (MVC) y uso de getters y setters para aplicar encapsulamiento explícitamente.
- **Iteración 05:** implementar relaciones de dependencia y asociación entre clases de manera explícita a nivel de código por medio de un diagrama.

1. Objetivos

1.1 Objetivo General

Incorporar una sección de «Estadísticas» al juego de la Ruleta mediante el proceso de Round-trip [5], utilizando Visual Paradigm para gestionar el diagrama de clases del proyecto y estableciendo un vínculo bidireccional entre el modelo y el código.

1.2 Objetivos Específicos

1. Generar el diagrama UML del proyecto de la Ruleta mediante ingeniería inversa en Visual Paradigm.
2. Organizar y ajustar el diagrama generado, corrigiendo la disposición de las clases y eliminando elementos visuales innecesarios para mejorar la legibilidad.
3. Agregar la clase **Estadisticas** al diagrama UML, definiendo atributos, métodos y relaciones con otras clases del modelo. Para esto se deben considerar las siguientes métricas

a implementar:

- **totalJugadas:** cantidad de resultados registrados.
- **victorias y porcentajeVictorias.**
- **rachaMaxima:** mayor secuencia de aciertos consecutivos.
- **tipoMasJugado:** valor más frecuente de **TipoApuesta**.

Para guiar las decisiones de modelado, se plantean las siguientes preguntas de reflexión:

- **Relaciones:** ¿Con qué clases debe relacionarse **Estadisticas**?
 - **Tipo de relación:** ¿Conviene *asociación* o *dependencia* entre **Estadisticas** y cada clase?
 - **Cardinalidad:** ¿Cuál es la multiplicidad apropiada para cada relación?
 - **Operaciones y firmas:** Define las operaciones con tipos de retorno y parámetros
 - **Visibilidad y paquetes:** Establecer encapsulamientos (+/-) y el paquete destino.
4. Generar el código inicial de la clase **Estadisticas** a partir del diagrama UML, utilizando la opción **Generate Java Code** del proceso round-trip.
 5. Completar la implementación en Java con IntelliJ Idea.
 6. Actualizar el diagrama UML con el código desarrollado, aplicando el proceso de round-trip **Reverse Code** para que las modificaciones realizadas por el estudiante se reflejen en el modelo.
 7. Integrar la funcionalidad en la interfaz gráfica de la aplicación, incorporando en el menú una nueva sección denominada «Estadísticas» mediante Swing, que muestre los indicadores calculados.

2. Ayudas y Sugerencias

2.1 Ingeniería Inversa

Proceso mediante el cual, a partir de un código existente, se obtiene automáticamente un modelo UML, el cual refleja tanto las clases como sus relaciones y multiplicidades, visualizando de esta forma, la estructura del sistema sin necesidad de construir el diagrama manualmente.

2.1.1. Proceso

1. Ubicar la ruta absoluta del directorio java del proyecto, al este contener todas los paquetes y sus clases.

```
jesus@JesusTM ~/Desktop/Ruleta/src/main & main > ls  
java  
jesus@JesusTM ~/Desktop/Ruleta/src/main & main > ls java  
Controlador Launcher Modelo Vista  
jesus@JesusTM ~/Desktop/Ruleta/src/main & main > pwd  
/home/jesus/Desktop/Ruleta/src/main  
jesus@JesusTM ~/Desktop/Ruleta/src/main & main > 
```

Figura 1: Ruta Proyecto

2. Abrir Visual Paradigm y crear un nuevo proyecto o abrir uno existente.
3. En el menú superior seleccionar: **Tools** → **Code** → **Reverse Java Code**.

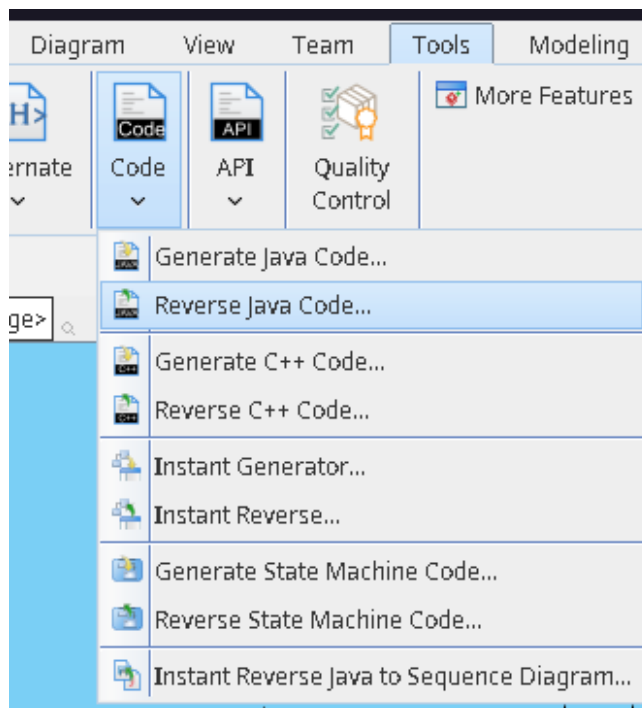


Figura 2: Opción dentro de Visual Paradigm

4. En la ventana emergente, especificar la ruta de la carpeta que contiene los archivos .java.

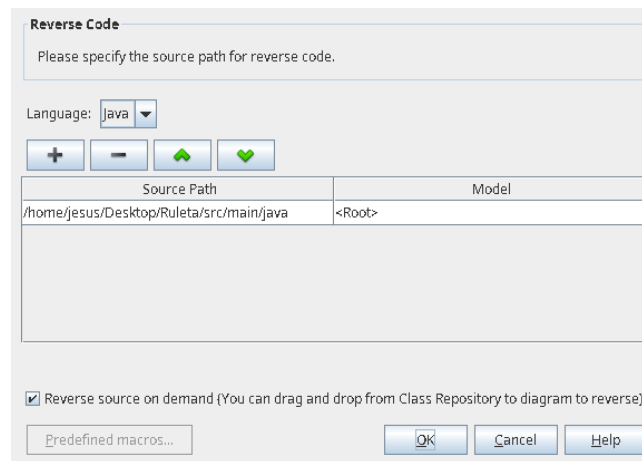


Figura 3: Ingresar Ruta Proyecto

5. Finalmente se generará un diagrama de clases que incluye las clases, relaciones y multiplicidades del código.

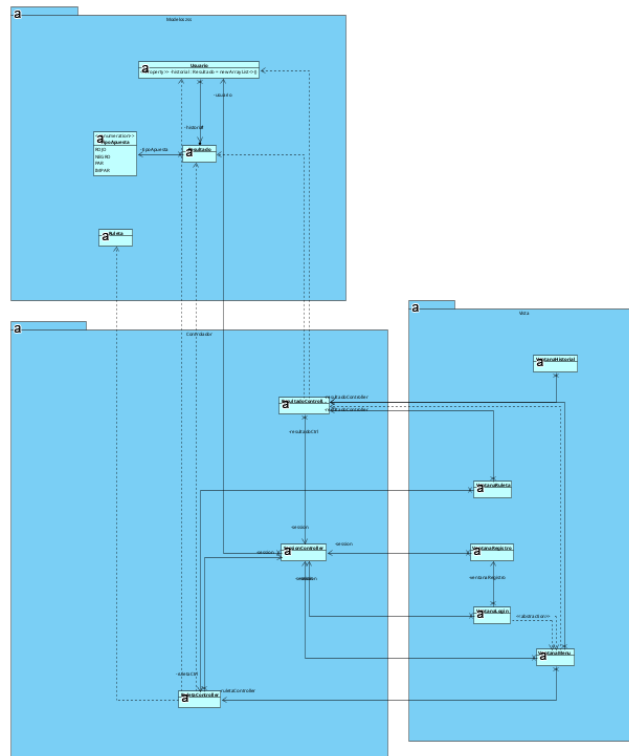


Figura 4: Diagrama de clases Generado

2.2 Ingeniería Directa

Proceso en el que, a partir de un modelo UML, se genera el código equivalente, trasladando las definiciones del diagrama, como clases, atributos y métodos, a una implementación inicial a nivel de código.

2.2.1. Proceso

1. Dentro de Visual Paradigm, se debe crear un diagrama de de clases
2. Con el diagrama ya generado, en el menú superior: **Tools** → **Code** → **Generate Java Code**.

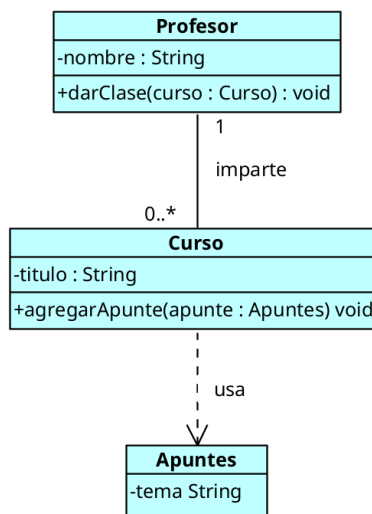


Figura 5: Diagrama de Clases

3. Dentro de la ventana se debe indicar la ubicación donde se quiere generar el código.

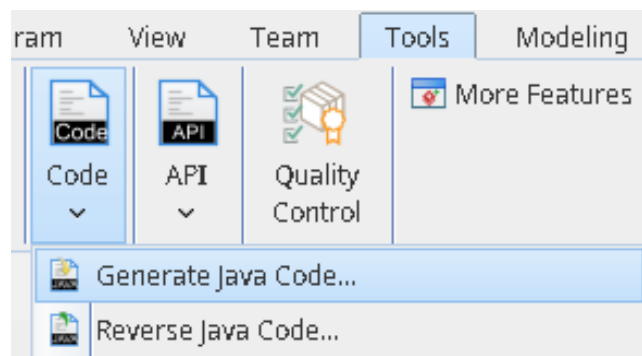


Figura 6: Generar Código - 1

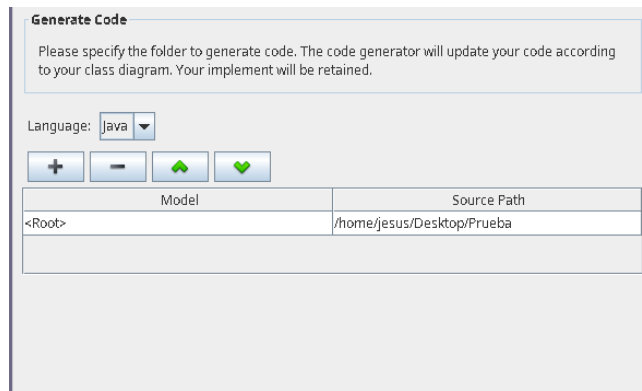


Figura 7: Generar Código - 2

4. Finalmente el código es generado.

```
jesus@JesusTM ~/Desktop/Prueba > ls
Apuntes.java  Curso.java  Profesor.java
jesus@JesusTM ~/Desktop/Prueba > bat Apuntes.java
public class Apuntes {

    private int tema;

}
jesus@JesusTM ~/Desktop/Prueba > bat Curso.java
public class Curso {

    private String titulo;

    /**
     *
     * @param apunte
     */
    public void agregarApunte(Apuntes apunte) {
        // TODO - implement Curso.agregarApunte
        throw new UnsupportedOperationException();
    }

}
jesus@JesusTM ~/Desktop/Prueba > bat Profesor.java
public class Profesor {

    private String nombre;

    /**
     *
     * @param curso
     */
    public void darClase(Curso curso) {
        // TODO - implement Profesor.darClase
        throw new UnsupportedOperationException();
    }

}
```

Figura 8: Código

2.3 Sincronización (Round-Trip) [5]

Alternativamente a estos dos procesos, existe la posibilidad de establecer un vínculo bidireccional entre una herramienta de modelado como Visual Paradigm y un IDE, la cual permita mantener el código y el diagrama alineados en ambos sentidos.

El proceso de Round-trip (ya sea Reverse Code o Generate Code) solo puede realizarse si previamente se configuró la relación entre el código fuente y el diagrama en Visual Paradigm mediante la opción: **Tools** → **Code** → **Reverse Java Code...**, indicando la carpeta **java** del proyecto como **Source Path**.

- Al momento de realizar cambios en el código, es necesario ejecutar ingeniería inversa para que estos se reflejen en el modelo UML.
 1. Realizar una modificación en el código Java del proyecto (por ejemplo, agregar un nuevo atributo o método en alguna clase).
 2. Abrir el diagrama de clases correspondiente en Visual Paradigm.
 3. Hacer clic derecho sobre el área en blanco del diagrama.
 4. Seleccionar: **Utilities** → **Java Round-trip** → **Reverse Code**.
 5. El diagrama se sincronizará automáticamente, incorporando los cambios realizados en el código Java.
- Cuando se realizan cambios en el modelo, es necesario volver a generar código utilizando opciones como merge, de manera que se mantenga lo existente y se agregue lo nuevo.
 1. Realizar una modificación en el diagrama UML en Visual Paradigm (por ejemplo, agregar una nueva clase, atributo o método en el Class Diagram).
 2. Abrir el diagrama de clases correspondiente.
 3. Hacer clic derecho sobre el área en blanco del diagrama.
 4. Seleccionar: **Utilities** → **Java Round-trip** → **Generate Code**.
 5. Visual Paradigm actualizará el código Java en la carpeta vinculada, generando los nuevos elementos (clases, atributos o métodos) y manteniendo el resto del código existente.

2. Referencias

[1] [Documentación] Apuntes Java - Jesús. <https://jessustm.github.io/ICC490-1-P00/>

- [2] [Documentación] Apuntes List - Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/11.List/>
- [3] [Tutorial] Vincular un proyecto Java en IntelliJ IDEA con GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-01%5D%20GitHub%20-%20IntelliJIDEA.pdf>
- [4] [Tutorial] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>
- [5] Java Round-Trip https://www.visual-paradigm.com/support/documents/vpuserguide/276/381_javaround-tr.html